

GITFLOW LAB — 5

Git • GitHub • VS Code • Branches • Merge • Conflict • Conflict Resolution

Name: G Venkata Varun

Rollno: 2420030187

Section: 6

1. Before You Start

Read one step at a time. First look at the screenshot. Then read the explanation. After that, perform the same action on your own computer.

2. Five Simple Words

Repository: Your project stored in Git.

Branch: A separate line of work.

Commit: A saved change in Git.

Push: Send your local changes to GitHub.

Merge: Bring changes from one branch into another branch.

3. Gitflow Used in This Lab

The practical uses a main branch, a develop branch, and feature branches.

```
main
├── develop
│   ├── feature/login
│   └── feature/conflict
```

The screenshots below show the complete work from creating the GitHub repository, setting up Git, creating branches, committing and pushing changes, creating a conflict, resolving it, and checking the result on GitHub.

Part A — Screenshot-Based Practical (Steps 1–58)

Each step below shows exactly what appears on screen, a simple explanation of what is happening, the Git command used (where applicable), and the corresponding screenshot from the practical session.

Step 1. Create the GitHub Repository

What you see: Open GitHub and create a new repository named Gitflow-Lab-5. In the screenshot, the repository is Public and the description is "For Task-5".

Simple explanation: This is the first step. The repository is the online place where the lab project will be stored.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * / Repository name *

Venkat-187 / Gitflow-Lab

✔ Gitflow-Lab is available.

Great repository names are short and memorable. How about [supreme-engine](#)?

Description

Task-5

6 / 350 characters

2 Configuration

Choose visibility * Choose who can see and commit to this repository

Public

Add README READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore .gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license Licenses explain how others can use your code. [About licenses](#)

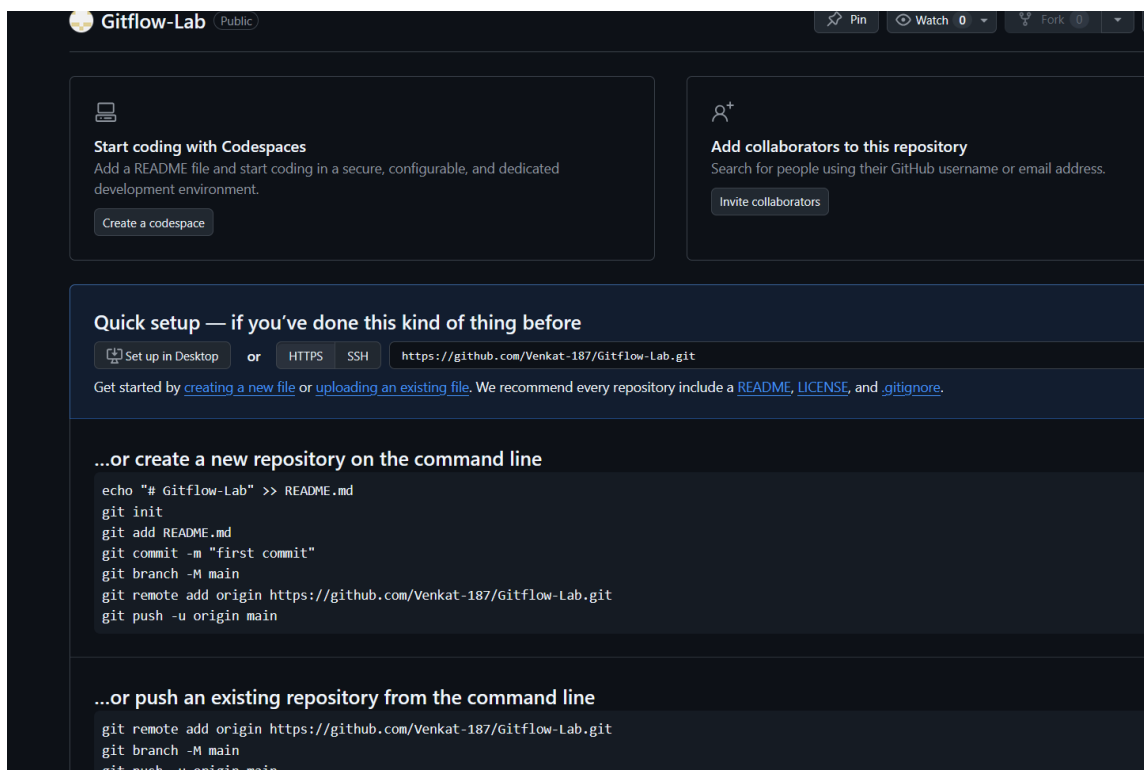
No license

Step 1

Step 2. Open the New Repository

What you see: After creating the repository, GitHub opens the Gitflow-Lab-5 repository page. The HTTPS repository address is visible.

Simple explanation: This page is the home page of the GitHub repository. We will connect the local project to this repository.



Step 2

Step 3. Understand the Clone Command

What you see: The terminal shows a clone command and a "Repository not found" message for the example address.

Simple explanation: A clone command copies a GitHub repository to the computer. The important idea is that the URL must point to an existing repository.

Step 4. Open the Project in VS Code

What you see: The VS Code Explorer shows the FSD workspace and the project folders.

Simple explanation: VS Code is used to work with the files and to run Git commands from the terminal.

Step 5. Move to the Project Folder

What you see: The PowerShell terminal moves into D: FSD and then into the Gitflow-Lab-5 folder.

Simple explanation: Git commands should be run inside the project folder.

Command:

```
cd Gitflow-Lab-5
```

Purpose: Moves the terminal into the project folder.

```

Problems  Output  Debug Console  Terminal  Ports

...
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 5

Step 6. Create README.md

What you see: README.md is created and git status shows it as an untracked file on the main branch.

Simple explanation: README.md is a simple file used to keep information about the project. Git is now detecting it as a new file.

Command:

```
git status
```

Purpose: Checks the current branch and file status.

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status

Untracked files:
  (use "git add <file>..." to include in what will be committed)
      README.md

nothing added to commit but untracked files present (use "git add" to track)
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 6

Step 7. Stage README.md

What you see: The command git add README.md is entered.

Simple explanation: git add puts the selected file into the staging area. Staging means the file is ready to be saved in a commit.

Command:

```
git add README.md
```

Purpose: Stages README.md for the next commit.

```

Problems Output Debug Console Terminal Ports

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add README.md
❖ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 7

Step 8. Check the Staged File

What you see: git status shows README.md under "Changes to be committed".

Simple explanation: This confirms that README.md has been staged successfully.

Command:

```
git status
```

Purpose: Confirms that README.md is staged.

```

Problems Output Debug Console Terminal Ports

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md

○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 8

Step 9. Make the Initial Commit

What you see: The terminal shows the initial commit being created successfully.

Simple explanation: git commit saves the staged changes in Git history. The first saved version is the initial commit.

Command:

```
git commit -m "Initial commit"
```

Purpose: Saves the first project version.

```

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Initial commit"
[master (root-commit) 6c635e2] Initial commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 9

Step 10. Check the Remote Command

What you see: The terminal shows Git's usage/error message after an incorrect remote command.

Simple explanation: A Git command must be written in the correct format. If Git shows usage information, check the command before continuing.

Step 11. Check the GitHub Remote

What you see: `git remote -v` shows the origin repository address.

Simple explanation: origin is the short name used for the GitHub repository connected to the local project.

Command:

```
git remote -v
```

Purpose: Shows the GitHub repository connected as origin.

```
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git remote -v
  origin https://github.com/Venkat-187/Gitflow-Lab.git (fetch)
  origin https://github.com/Venkat-187/Gitflow-Lab.git (push)
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> █
```

Step 11

Step 12. Check the Commit History

What you see: The terminal shows `git log --oneline` with the Initial commit.

Simple explanation: `git log --oneline` gives a short view of the saved commits.

Command:

```
git log --oneline
```

Purpose: Shows a short list of commits.

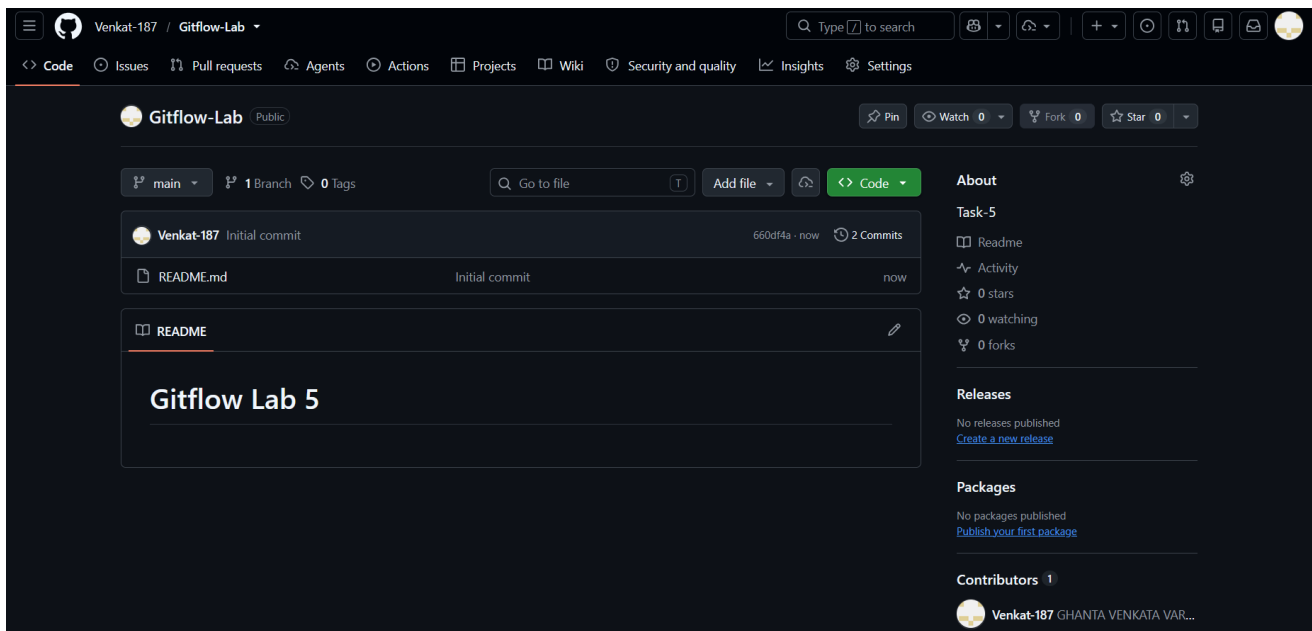
```
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git log --oneline
  6c635e2 (HEAD -> main) Initial commit
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> █
```

Step 12

Step 13. Check the Repository on GitHub

What you see: GitHub shows the main branch, README.md, and the Initial commit.

Simple explanation: This confirms that the repository exists on GitHub and the initial project content is visible.



Step 13

Step 14. Check Git Remote Carefully

What you see: The terminal again shows Git's remote command usage/error information.

Simple explanation: If the command is entered incorrectly, Git explains the correct usage. Follow the command format carefully.

```

1 FILE CHANGED; 1 INSERTION(1)
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 447 bytes | 149.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Venkat-187/Gitflow-Lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git remote -v
origin https://github.com/Venkat-187/Gitflow-Lab.git (fetch)
origin https://github.com/Venkat-187/Gitflow-Lab.git (push)
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 14

Step 15. Create the develop Branch

What you see: The command `git checkout -b develop` creates and switches to the develop branch. `git branch` lists develop and main.

Simple explanation: develop is the branch used for development work before changes are moved to main.

Command:

```
git checkout -b develop
```

Purpose: Creates and switches to develop.

```
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git branch
* develop
  main
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> |
```

Step 15

Step 16. Push develop to GitHub

What you see: The terminal shows git push -u origin develop completing successfully.

Simple explanation: The -u option connects the local develop branch with the remote develop branch on GitHub.

Command:

```
git push -u origin develop
```

Purpose: Uploads develop to GitHub and sets its upstream.

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
(use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/Venkat-187/Gitflow-Lab/pull/new/develop
remote:
To https://github.com/Venkat-187/Gitflow-Lab.git
 * [new branch]      develop -> develop
```

Step 16

Step 17. Create feature/login

What you see: The terminal switches from develop to a new feature/login branch.

Simple explanation: A feature branch is used for one particular feature. Here the feature is login.

Command:

```
git checkout -b feature/login
```

Purpose: Creates and switches to feature/login.

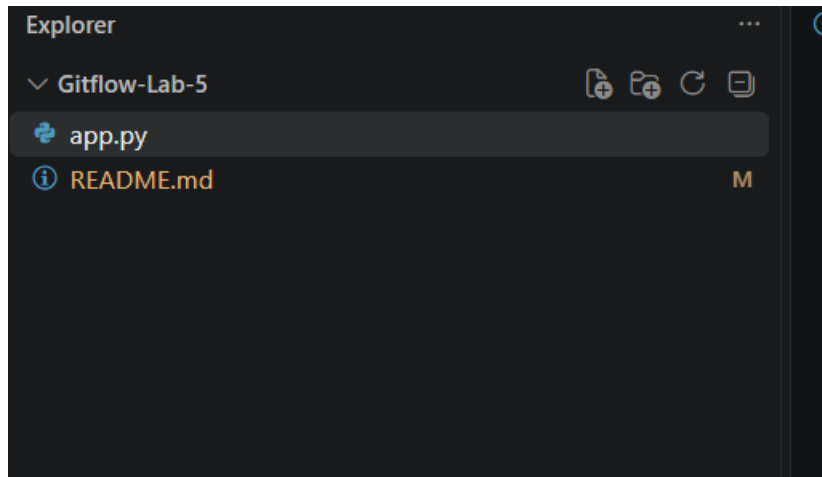
```
branch develop set up to track origin/develop .
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout -b feature/login
Switched to a new branch 'feature/login'
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> |
```

Step 17

Step 18. Create app.py

What you see: The VS Code Explorer shows app.py as a new file in Gitflow-Lab-5.

Simple explanation: app.py will contain the small Python program used in this lab.

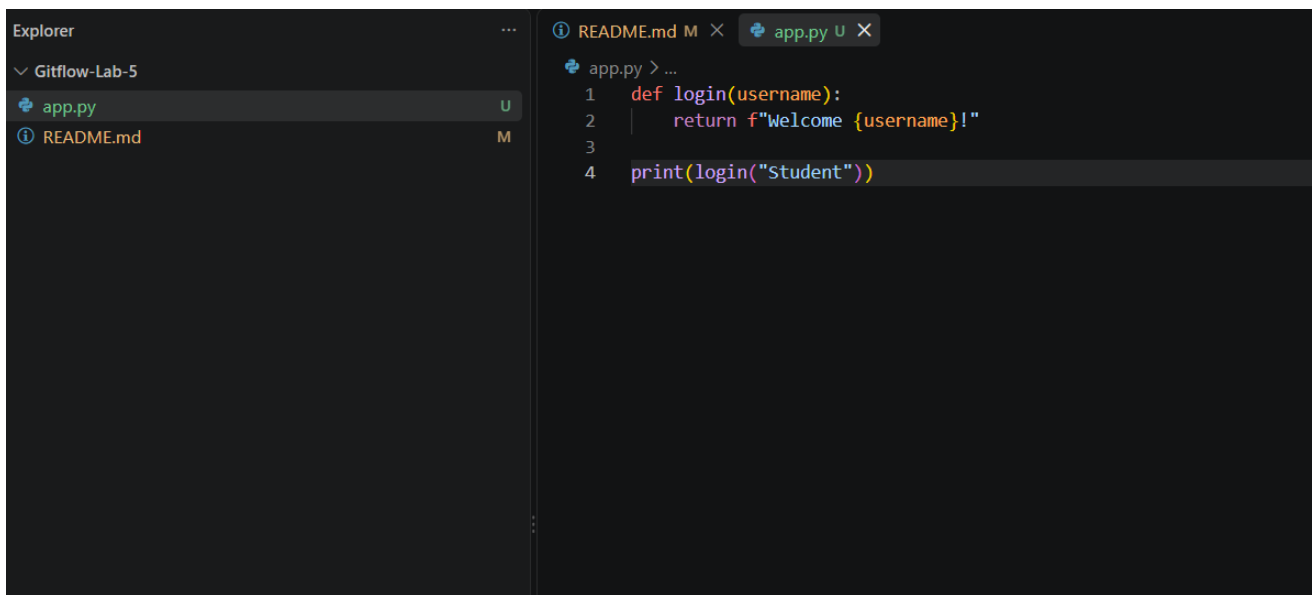


Step 18

Step 19. Write the Login Program

What you see: The editor shows the login function and `print(login("Student"))` in app.py.

Simple explanation: The program receives a username and returns a welcome message.



Step 19

Step 20. Check the Python Command

What you see: The terminal shows an error after app.py was entered directly as a PowerShell command.

Simple explanation: A Python file should be run using Python, for example: `python app.py`.

```

Explorer
  Gitflow-Lab-5
    app.py U
    README.md M

app.py
1 def login(username):
2     return f"Welcome {username}!"
3
4 print(login("Student"))

Terminal
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> python app.py
Welcome Student!
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 20

Step 21. Run app.py

What you see: The terminal shows `python app.py` producing "Welcome Student", followed by `git status` showing `app.py` as untracked.

Simple explanation: The program is working. Git also knows that `app.py` is a new file that has not yet been staged.

Command:

```
python app.py
```

Purpose: Runs the Python program.

```

Terminal
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch feature/login
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app.py

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

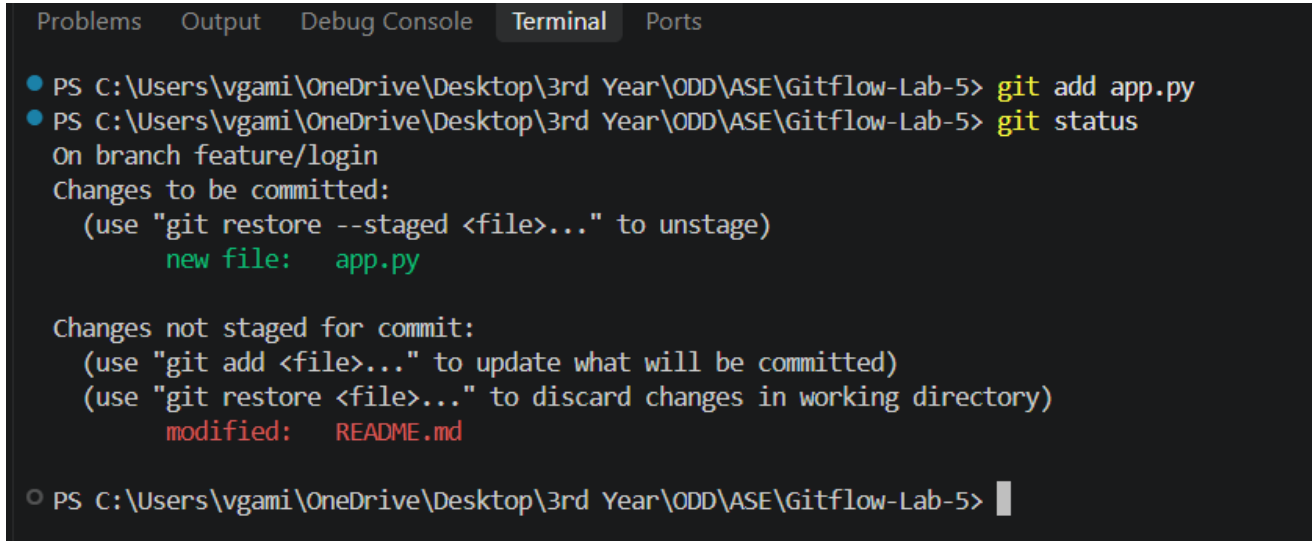
```

Step 21

Step 22. Check the New File Again

What you see: The terminal again shows python app.py producing "Welcome Student" and git status showing app.py as untracked.

Simple explanation: This confirms both the program output and the Git status.



```

Problems  Output  Debug Console  Terminal  Ports

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add app.py
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch feature/login
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   app.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 22

Step 23. Commit the Login Feature

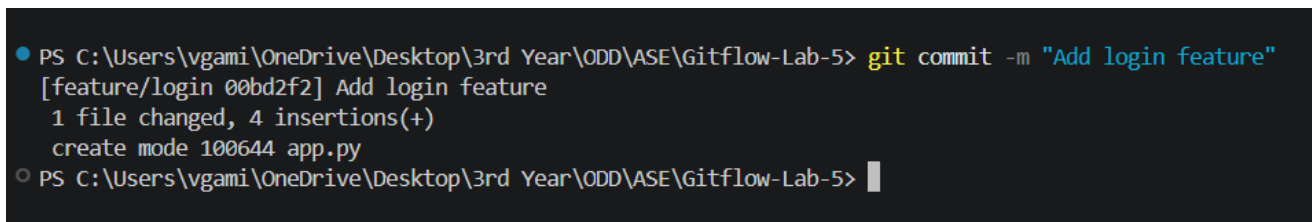
What you see: The command git commit -m "Add login feature" creates the feature/login commit.

Simple explanation: The commit message tells us what was added. This saves the login feature in Git history.

Command:

```
git commit -m "Add login feature"
```

Purpose: Saves the login feature in Git history.



```

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Add login feature"
[feature/login 00bd2f2] Add login feature
 1 file changed, 4 insertions(+)
 create mode 100644 app.py
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 23

Step 24. Stage app.py

What you see: The terminal shows `git add app.py` and `git status` with `app.py` staged.

Simple explanation: The file is now ready to be included in the next commit.

Command:

```
git add app.py
```

Purpose: Stages `app.py`.

```
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git log --oneline
00bd2f2 (HEAD -> feature/login) Add login feature
660df4a (origin/main, origin/develop, main, develop) Initial commit
6c635e2 Initial commit
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>
```

Step 24

Step 25. Check and Push feature/login

What you see: The terminal shows the Add login feature commit, the short log, and the `feature/login` push.

Simple explanation: This step checks the feature history and sends the feature branch to GitHub.

Command:

```
git push -u origin feature/login
```

Purpose: Uploads `feature/login` to GitHub.

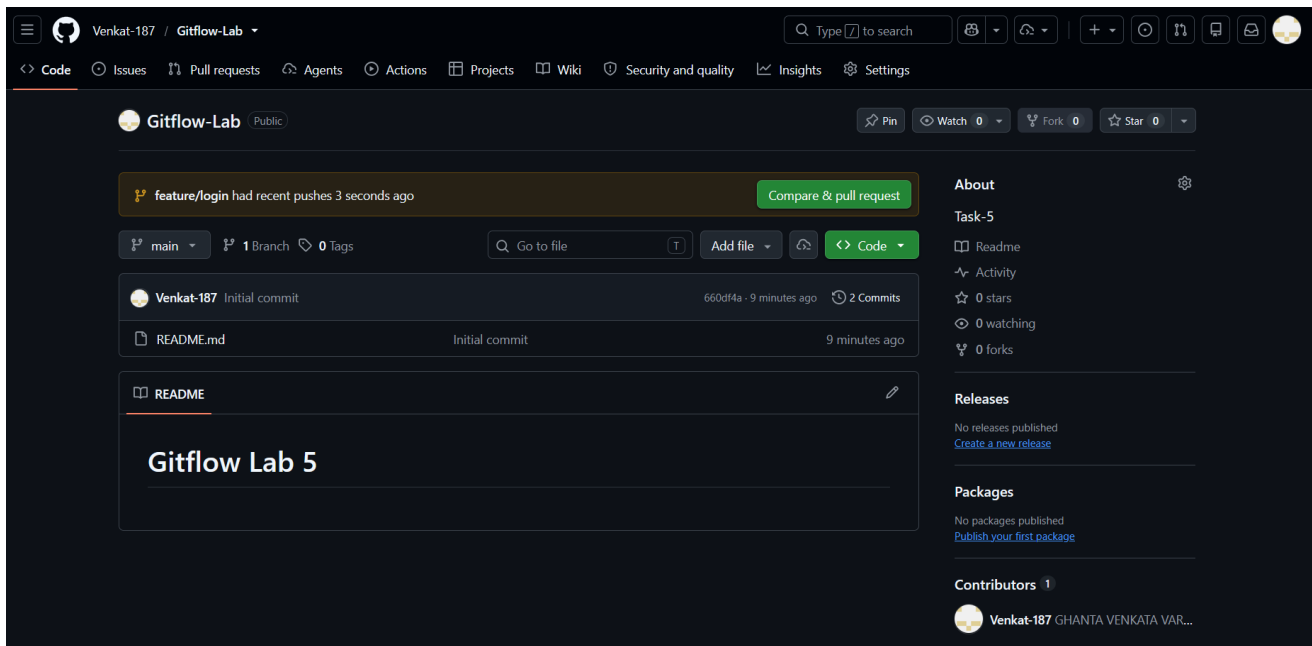
```
Problems Output Debug Console Terminal Ports
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push -u origin feature/login
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 344 bytes | 344.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/login' on GitHub by visiting:
remote:   https://github.com/Venkat-187/Gitflow-Lab/pull/new/feature/login
remote:
To https://github.com/Venkat-187/Gitflow-Lab.git
 * [new branch]      feature/login -> feature/login
branch 'feature/login' set up to track 'origin/feature/login'.
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>
```

Step 25

Step 26. See feature/login on GitHub

What you see: The GitHub repository page shows a recent push for `feature/login`.

Simple explanation: This confirms that the feature branch has been uploaded to GitHub.

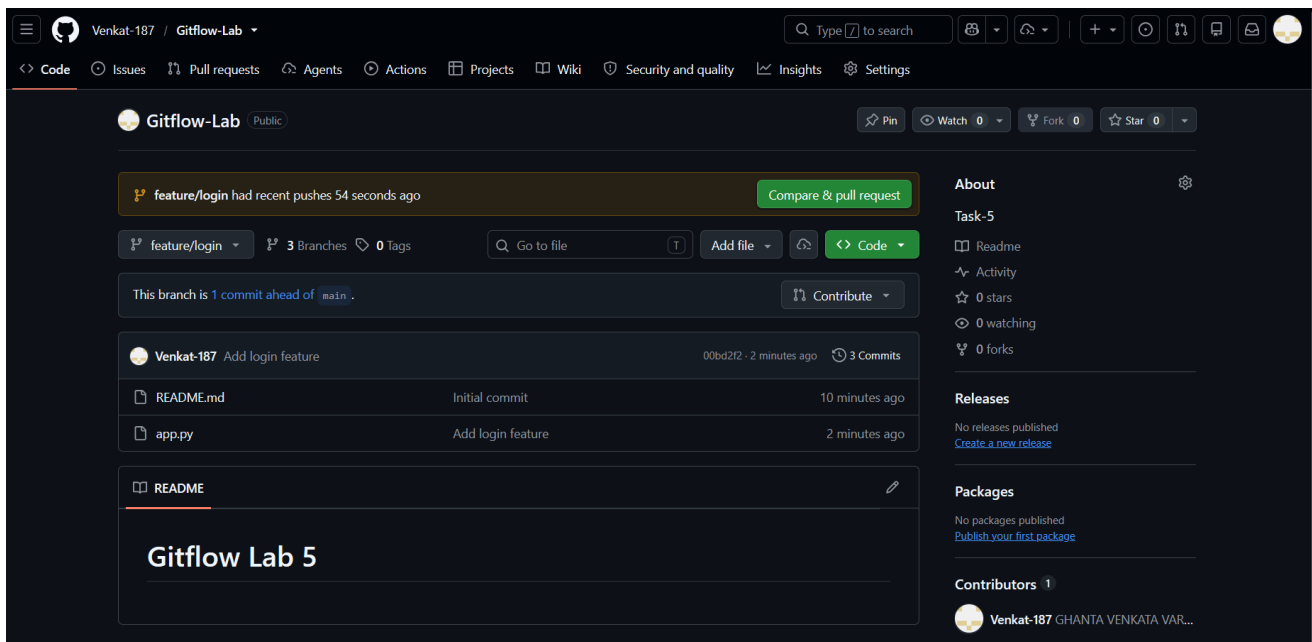


Step 26

Step 27. Open feature/login Branch

What you see: GitHub shows feature/login with app.py and README.md and indicates that the branch is ahead of main.

Simple explanation: The feature branch contains the login work separately from main.



Step 27

Step 28. Review feature/login History

What you see: The terminal shows the staged app.py, the Add login feature commit, and the Git history.

Simple explanation: This is a quick check that the feature work was saved correctly.

Command:

```
git log --oneline
```

Purpose: Shows the short commit history.

Step 29. Finish the feature/login Push

What you see: The terminal shows the push progress and successful upload to GitHub.

Simple explanation: The remote feature/login branch is now updated.

Step 30. Merge feature/login into develop

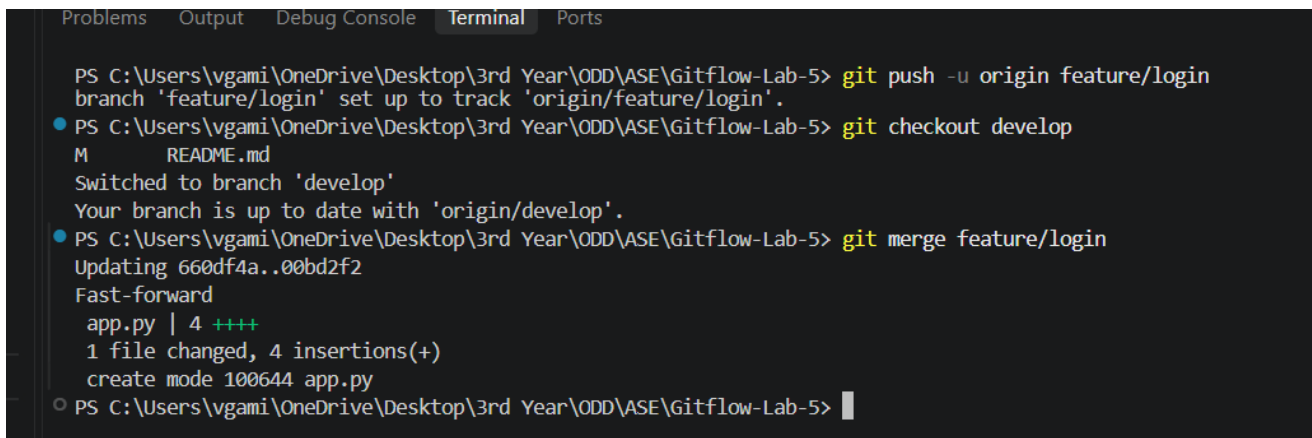
What you see: The terminal shows git merge feature/login on develop and the message Fast-forward.

Simple explanation: The login feature is now included in develop. Fast-forward means Git could move develop forward without creating a separate merge commit.

Command:

```
git merge feature/login
```

Purpose: Combines feature/login into the current develop branch.



```

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push -u origin feature/login
branch 'feature/login' set up to track 'origin/feature/login'.
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout develop
M      README.md
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git merge feature/login
Updating 660df4a..00bd2f2
Fast-forward
 app.py | 4 +++
1 file changed, 4 insertions(+)
 create mode 100644 app.py
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 30

Step 31. Push Updated develop

What you see: The terminal shows git log --oneline and git push origin develop completing successfully.

Simple explanation: The updated develop branch is now available on GitHub.

Command:

```
git push origin develop
```

Purpose: Uploads the updated develop branch.

```

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git log --oneline
00bd2f2 (HEAD -> develop, origin/feature/login, feature/login) Add login feature
660df4a (origin/main, origin/develop, main) Initial commit
6c635e2 Initial commit

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Venkat-187/Gitflow-Lab.git
660df4a..00bd2f2 develop -> develop

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 31

Step 32. Check GitHub Branches

What you see: GitHub shows main, develop, and feature/login in the branch list.

Simple explanation: This is the Gitflow branch structure created so far.

The screenshot shows the GitHub 'Branches' page for the repository 'Venkat-187 / Gitflow-Lab'. The 'Overview' tab is selected, showing a search bar and three sections: 'Default', 'Your branches', and 'Active branches'. Each section contains a table of branches with columns for Branch, Updated, Check status, Behind/Ahead, and Pull request.

Branch	Updated	Check status	Behind	Ahead	Pull request
main	14 minutes ago				Default
develop	now		0	1	
feature/login	5 minutes ago		0	1	

Step 32

Step 33. Prepare the Conflict Branch

What you see: The terminal shows branch history, a develop push, the branch list, and creation of feature/conflict.

Simple explanation: feature/conflict will be used to demonstrate a merge conflict.

Command:

```
git checkout -b feature/conflict
```

Purpose: Creates and switches to feature/conflict.

```

Problems Output Debug Console Terminal Ports

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git merge feature/login
create mode 100644 app.py
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git log --oneline
00bd2f2 (HEAD -> develop, origin/feature/login, feature/login) Add login feature
660df4a (origin/main, origin/develop, main) Initial commit
6c635e2 Initial commit
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Venkat-187/Gitflow-Lab.git
660df4a..00bd2f2 develop -> develop
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout -b feature/conflict
Switched to a new branch 'feature/conflict'
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

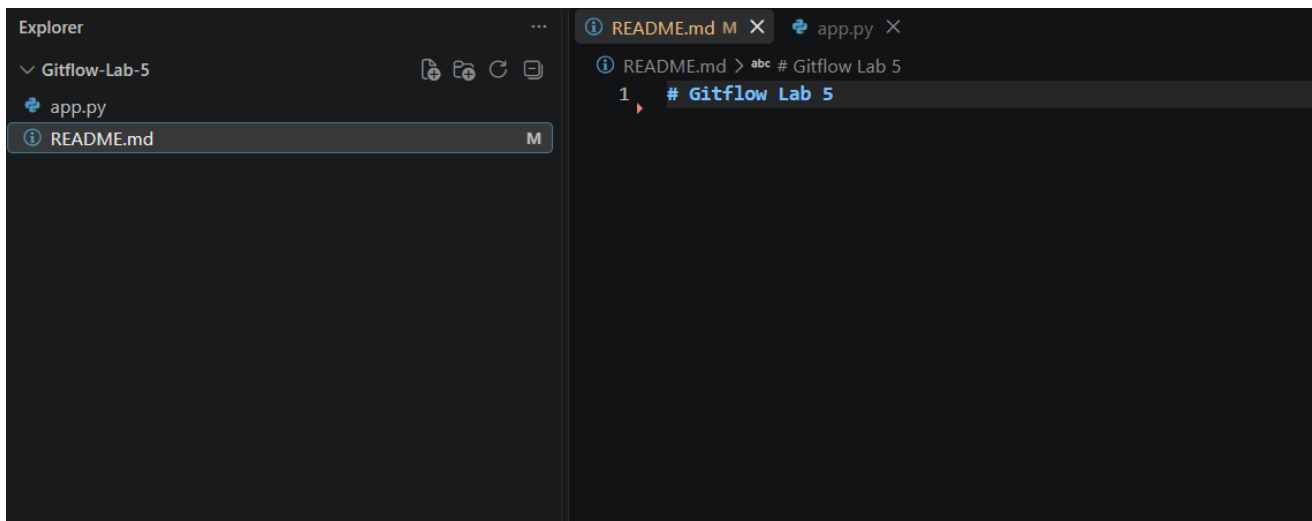
```

Step 33

Step 34. Open README.md

What you see: VS Code shows README.md with the Gitflow Lab 5 heading while working on the conflict branch.

Simple explanation: README.md will be changed on different branches so Git has two different versions to compare.

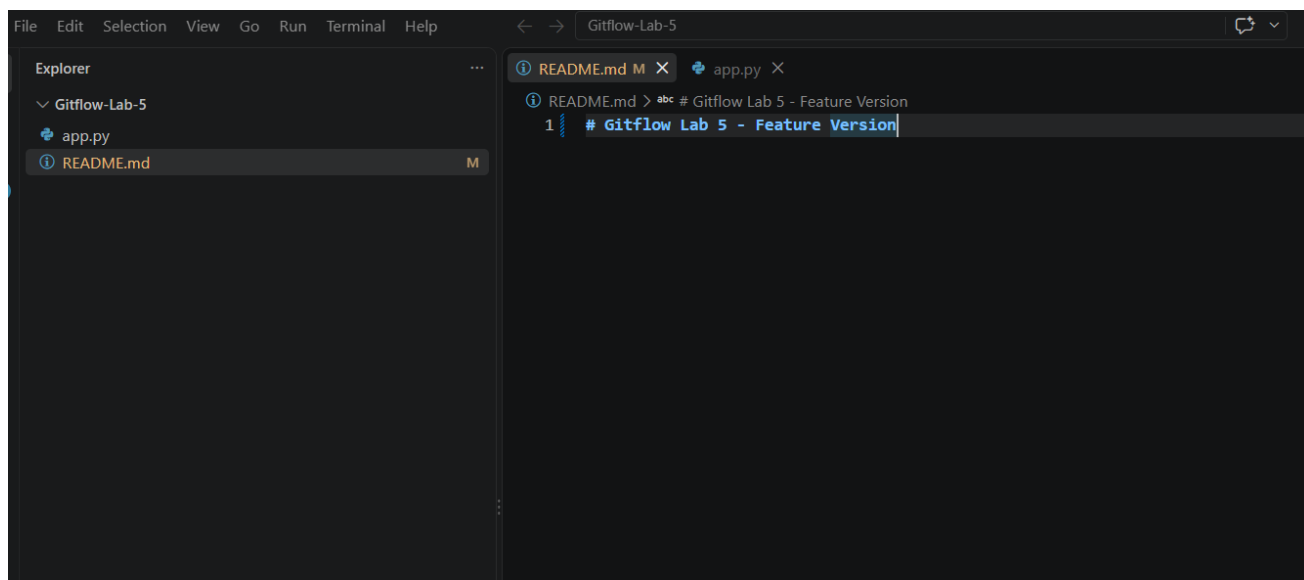


Step 34

Step 35. Change the Feature Version

What you see: README.md shows "Gitflow Lab 5 - Feature Version".

Simple explanation: This creates the feature-side version of the README file.



Step 35

Step 36. Switch to feature/conflict

What you see: The terminal shows git checkout -b feature/conflict and confirms the branch switch.

Simple explanation: The conflict work is being done on this feature branch.

```

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout -b feature/conflict
Switched to a new branch 'feature/conflict'
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch feature/conflict
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

```

Step 36

Step 37. Stage README.md on feature/conflict

What you see: git add README.md is used and git status shows README.md staged.

Simple explanation: The feature-side README change is ready to commit.

Command:

```
git add README.md
```

Purpose: Stages the feature-side README change.

```

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add README.md
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch feature/conflict
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 37

Step 38. Commit the Conflict Feature

What you see: The terminal shows the commit command for the conflict feature.

Simple explanation: The feature-side README version is saved in Git history.

Command:

```
git commit -m "Update README for conflict feature"
```

Purpose: Saves the feature-side README change.

```

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch feature/conflict
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Update README for conflict feature"
[feature/conflict c8064f2] Update README for conflict feature
 1 file changed, 1 insertion(+), 1 deletion(-)
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 38

Step 39. Confirm the Conflict Feature Commit

What you see: The terminal shows the successful "Update README for conflict feature" commit.

Simple explanation: The feature/conflict branch now has its own README change.

```

1 file changed, 1 insertion(+), 1 deletion(-)
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git branch
  develop
* feature/conflict
  feature/login
  main
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

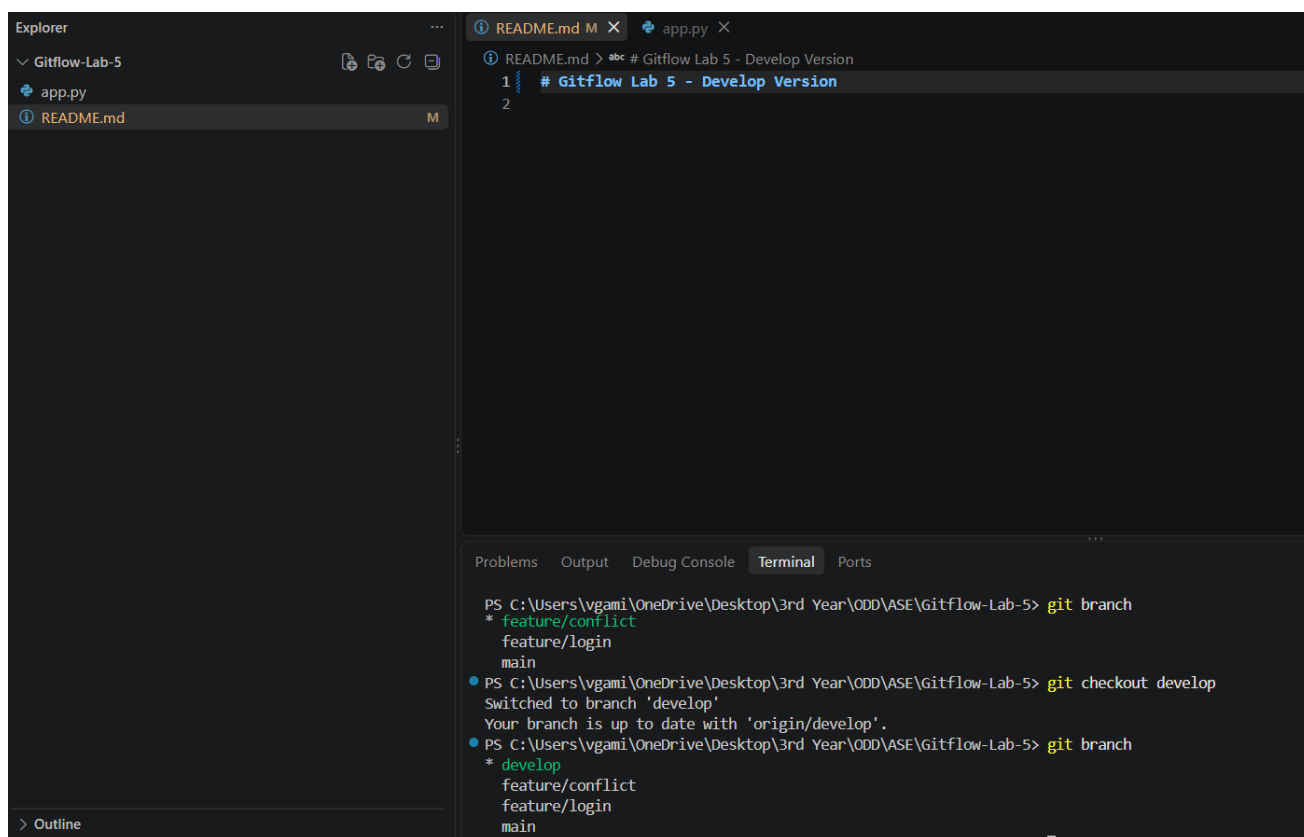
```

Step 39

Step 40. Create the Develop Version

What you see: README.md shows "Gitflow Lab 5 - Develop Version".

Simple explanation: Now a different README version is prepared on develop.

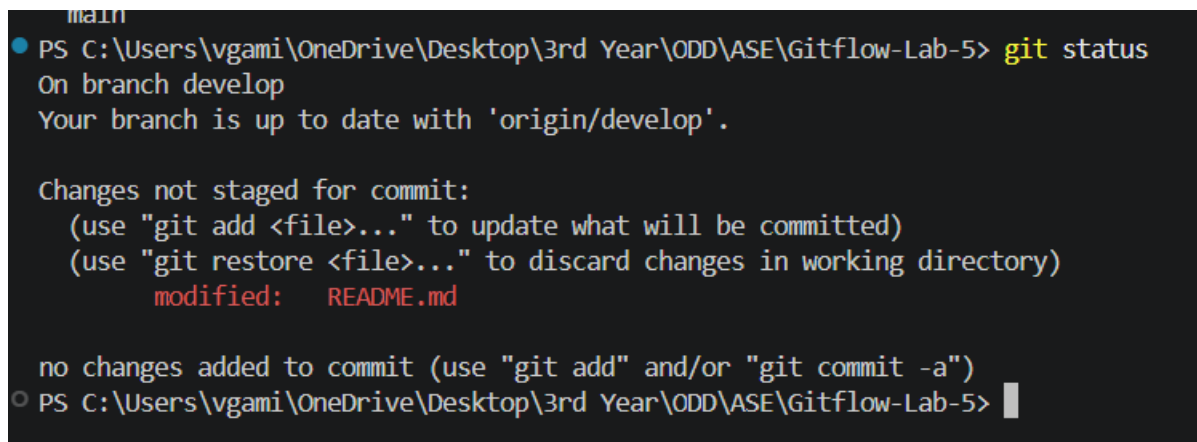


Step 40

Step 41. Review the Branch History

What you see: VS Code Source Control/Timeline shows develop, feature/conflict, feature/login, and main in the history.

Simple explanation: The history helps us see that different branches contain different changes.



Step 41

Step 42. Review the Project History

What you see: VS Code shows the Gitflow-Lab-5 project and the recorded README changes.

Simple explanation: The Timeline is useful for seeing saved file changes over time.

Step 43. Continue the Develop Change

What you see: README.md again shows the develop version in VS Code.

Simple explanation: This is the develop-side README content that will be different from the feature version.

Step 44. Check develop Status

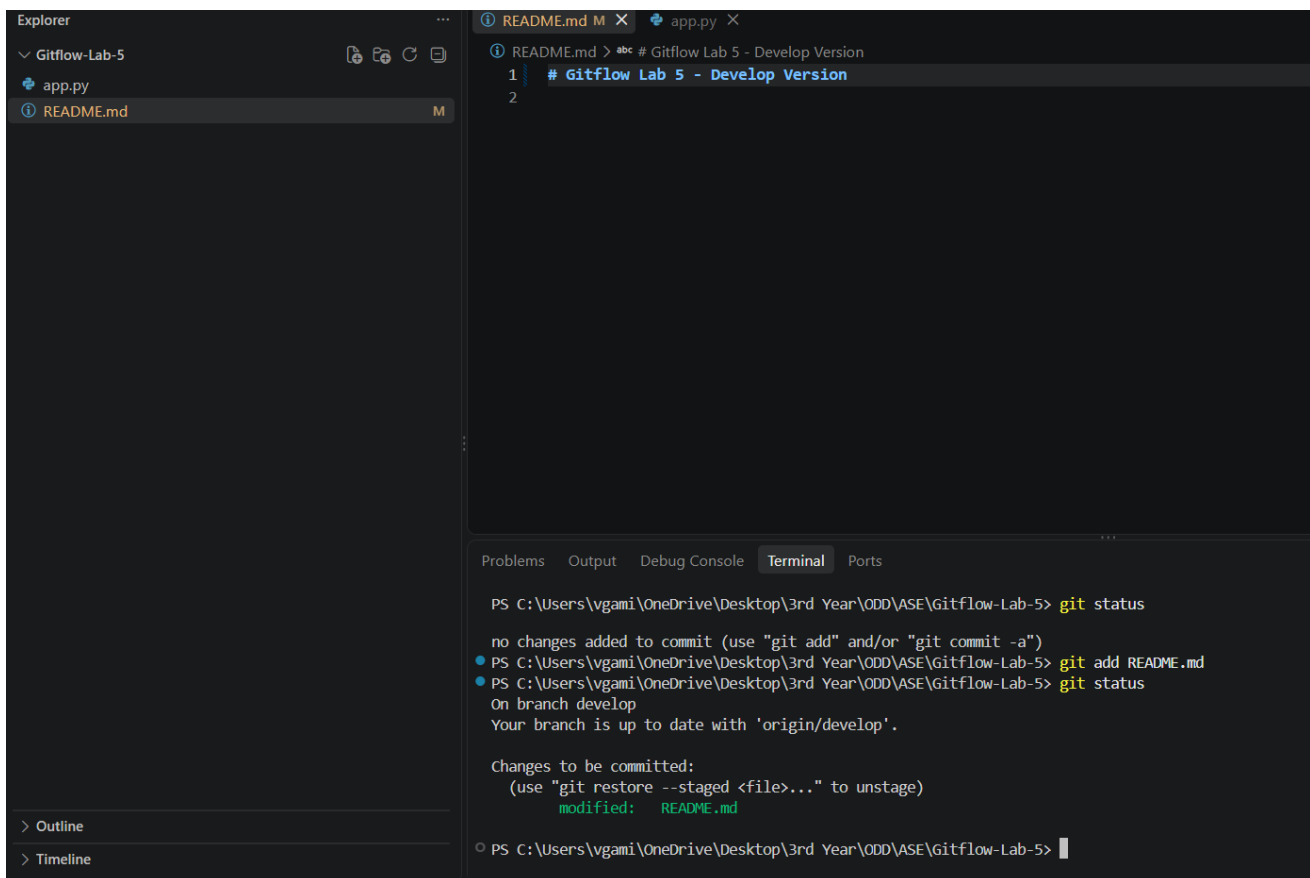
What you see: git status shows README.md modified on develop.

Simple explanation: Git has detected the change and it is ready to be staged.

Command:

```
git status
```

Purpose: Shows that README.md is modified.



The screenshot shows the VS Code interface. The Explorer pane on the left shows the file structure with 'app.py' and 'README.md' (marked with an 'M' for modified). The main editor shows the content of 'README.md', which includes the text '# Gitflow Lab 5 - Develop Version'. The Terminal pane at the bottom shows the output of the 'git status' command, indicating that 'README.md' is modified on the 'develop' branch and is ready to be committed.

```

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status

no changes added to commit (use "git add" and/or "git commit -a")
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add README.md
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch develop
Your branch is up to date with 'origin/develop'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>
  
```

Step 44

Step 45. Stage the Develop Change

What you see: VS Code Source Control shows the README.md change staged.

Simple explanation: The develop-side change is now ready for commit.

```

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch develop
Your branch is up to date with 'origin/develop'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

```

Step 45

Step 46. Commit the Develop README

What you see: The terminal shows git add README.md, git status, and the commit command.

Simple explanation: This saves the develop-side README change in Git history.

Command:

```
git commit -m "Update README on develop"
```

Purpose: Saves the develop-side README change.

```

• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Update README on develop"
[develop f51d614] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push origin develop
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 326 bytes | 326.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Venkat-187/Gitflow-Lab.git
   00bd2f2..f51d614  develop -> develop
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

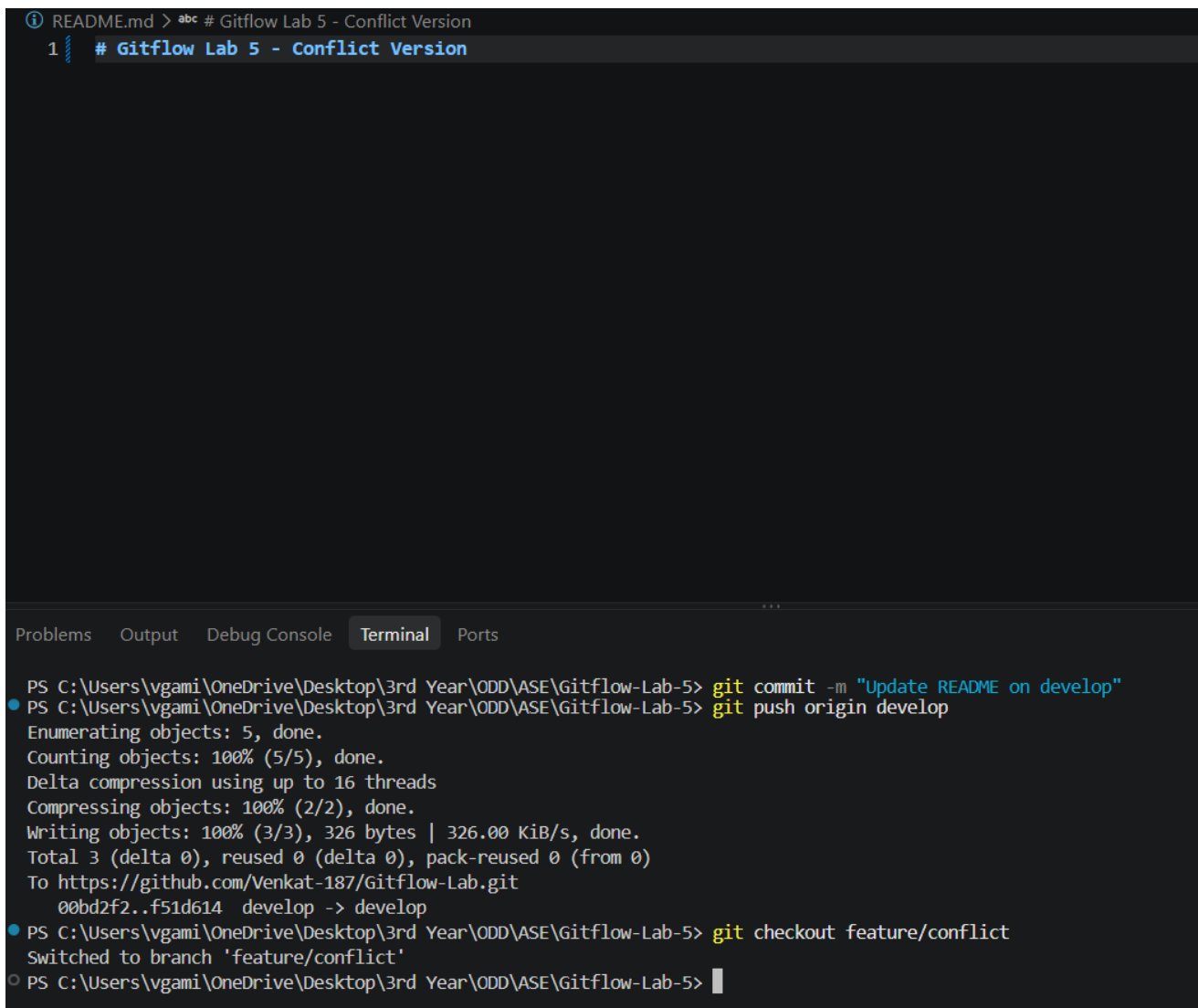
```

Step 46

Step 47. Create the Conflict Version

What you see: README.md shows "Gitflow Lab 5 - Conflict Version".

Simple explanation: This is another visible version of the same README file.



The screenshot shows the Visual Studio Code interface. The top bar indicates the active file is 'README.md' in the 'abc' workspace, with the title '# Gitflow Lab 5 - Conflict Version'. The file explorer on the left shows a single file named '1 # Gitflow Lab 5 - Conflict Version'. The bottom panel is the 'Terminal' tab, showing a PowerShell session with the following commands and output:

```

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Update README on develop"
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push origin develop
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 326 bytes | 326.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Venkat-187/Gitflow-Lab.git
   00bd2f2..f51d614  develop -> develop
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout feature/conflict
Switched to branch 'feature/conflict'
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 47

Step 48. Open the Merge State

What you see: VS Code shows README.md in the conflict workflow and Source Control information.

Simple explanation: The project is now ready for the merge that will produce the conflict.

The screenshot shows the VS Code interface with the Explorer sidebar on the left showing 'Gitflow-Lab-5' and 'app.py'. The main editor displays 'README.md' with the content: `# Gitflow Lab 5 - Conflict Version`. The Terminal at the bottom shows the following commands and output:

```
PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git push origin develop
To https://github.com/Venkat-187/Gitflow-Lab.git
00bd2f2..f51d614  develop -> develop
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout feature/conflict
Switched to branch 'feature/conflict'
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add README.md
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch feature/conflict
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>
```

Step 48

Step 49. Review the Conflict-Feature History

What you see: The VS Code Timeline shows the conflict-feature README update and saved files.

Simple explanation: The Timeline records the previous feature changes.

The screenshot shows the VS Code interface with the Explorer sidebar on the left showing 'Gitflow-Lab-5' and 'app.py'. The main editor displays 'README.md' with the content: `# Gitflow Lab 5 - Conflict Version`. The Terminal at the bottom shows the following commands and output:

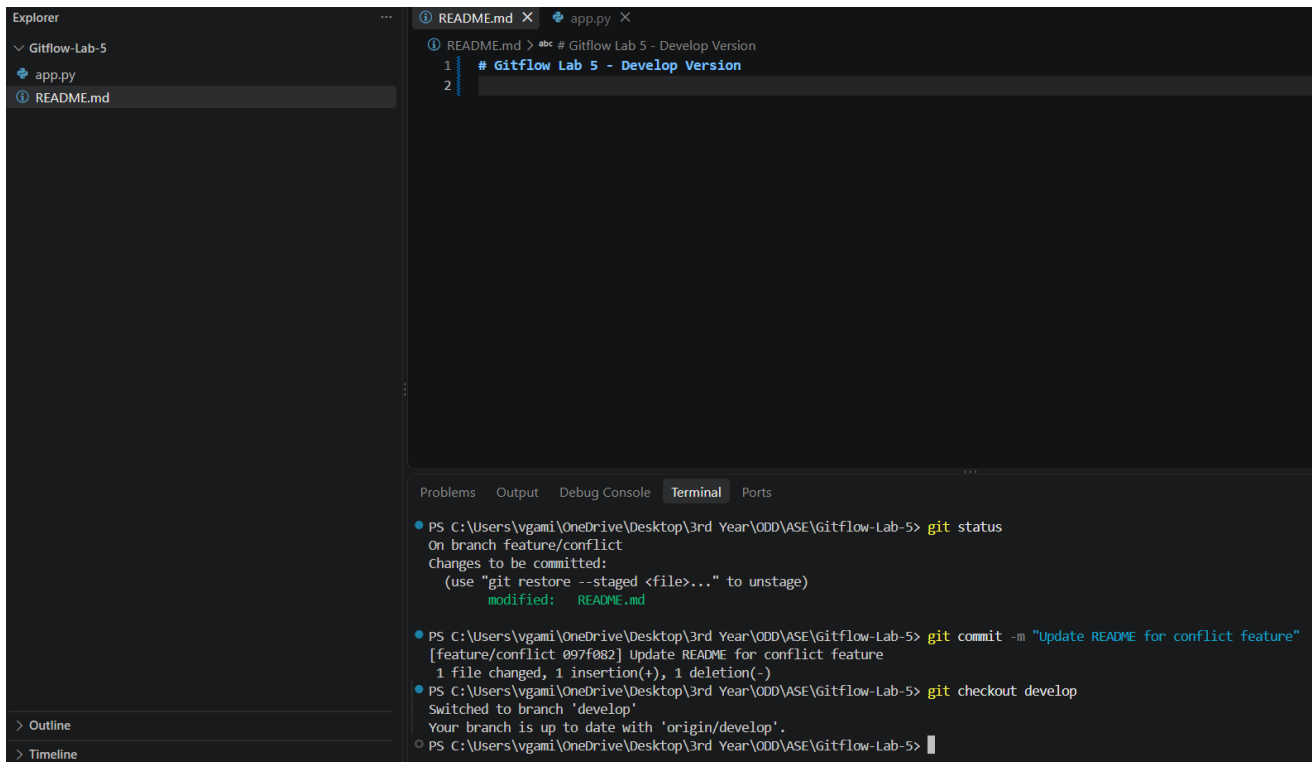
```
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout feature/conflict
Switched to branch 'feature/conflict'
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add README.md
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
On branch feature/conflict
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Update README for conflict feature"
[feature/conflict 097f082] Update README for conflict feature
 1 file changed, 1 insertion(+), 1 deletion(-)
• PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>
```

Step 49

Step 50. Review the Develop History

What you see: The VS Code Timeline shows the develop README update and saved files.

Simple explanation: The Timeline records the develop-side changes.



Step 50

Step 51. Create Another Develop Version

What you see: README.md shows "Gitflow Lab 5 - Another Develop Version".

Simple explanation: The develop branch now has a version that is different from the feature branch version.

```

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git status
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Update README for conflict feature"
[feature/conflict 097f082] Update README for conflict feature
 1 file changed, 1 insertion(+), 1 deletion(-)
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add README.md
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Update README on develop"
[develop 6c2a348] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
○ PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 51

Step 52. Open the Merge Editor

What you see: VS Code Merge Editor shows Incoming develop content and Current feature/conflict content for README.md.

Simple explanation: Git cannot automatically decide which version to keep. The Merge Editor shows both sides.

```

PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout develop
Your branch is up to date with 'origin/develop'.
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git add README.md
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git commit -m "Update README on develop"
[develop 6c2a348] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git checkout feature/conflict
Switched to branch 'feature/conflict'
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5> git merge develop
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
● PS C:\Users\vgami\OneDrive\Desktop\3rd Year\ODD\ASE\Gitflow-Lab-5>

```

Step 52

Step 53. See Both Versions Together

What you see: The Merge Editor shows the two README versions and the result area.

Simple explanation: Choose the correct content and make one final clean version in the result area.

Step 54. Complete the Conflict Resolution in Source Control

What you see: VS Code Source Control shows the merge state and the staged README.md.

Simple explanation: After the file is corrected, it must be staged and the merge must be completed with a commit.

Command:

```
git add README.md
```

Purpose: Stages the resolved README file.

Step 55. Review the Merge History

What you see: The terminal and Source Control show the merge history and Fast-forward/app.py information.

Simple explanation: This is a history check after the merge work.

Step 56. Review the Final Local History

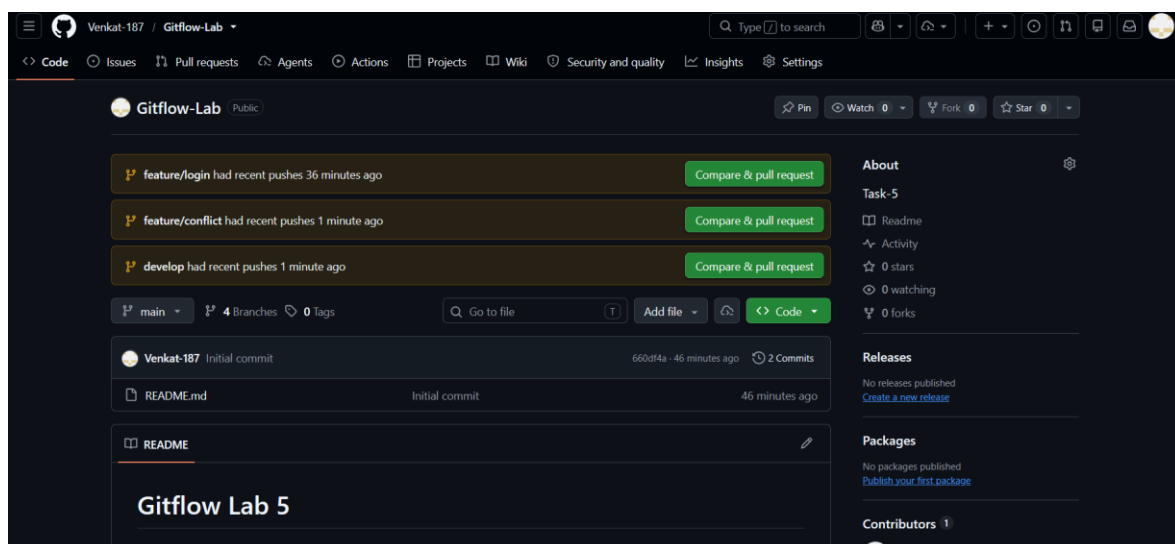
What you see: VS Code shows the project history after the conflict workflow.

Simple explanation: The history should now contain the work from the branches and the merge.

Step 57. Check GitHub Branches and Recent Pushes

What you see: GitHub shows recent pushes for feature/conflict and develop.

Simple explanation: This confirms that the branches have been pushed to GitHub.

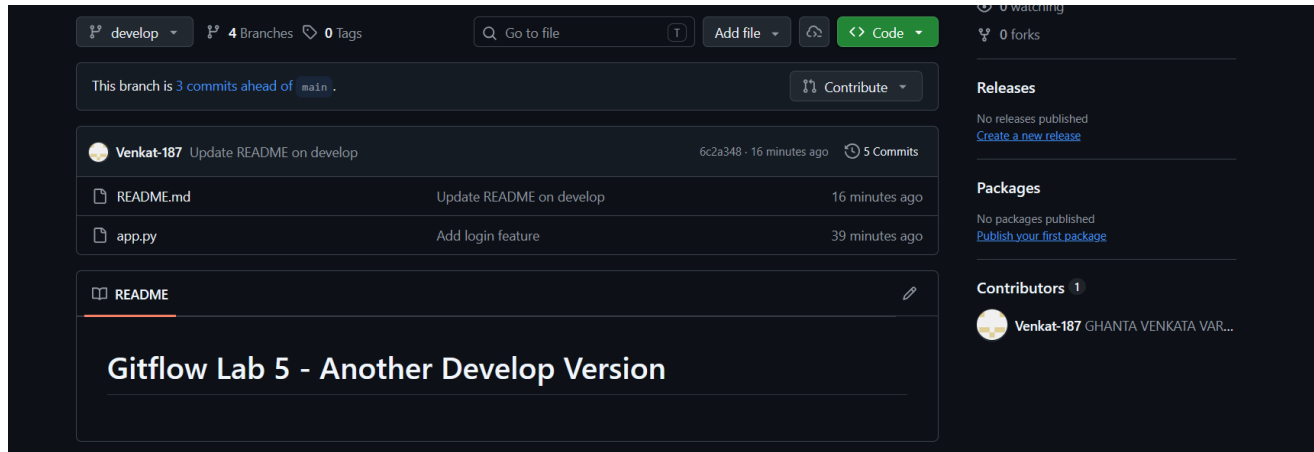


Step 57

Step 58. Open the Final develop Branch

What you see: GitHub shows the develop branch with README.md, app.py, the commit history, and the README content.

Simple explanation: This is the final verification screen. The develop branch contains the integrated project work.



Step 58

Final Revision

Remember the main flow: Create GitHub repository → connect the local project → create develop → create feature/login → commit → push → merge into develop → create feature/conflict → make different changes → merge → resolve the conflict → stage → commit → push → verify on GitHub.

Most Important Commands

- `git status` —
- `git branch` —
- `git checkout -b branch-name` —
- `git add file-name` — • `git commit -m "message"` —
- `git push` —
- `git merge branch-name` —
- `python app.py` —